Lab 06 Report: Using Monte Carlo Localization (MCL) for Sampling-based and Grid-based Path Planning and Following

Team 14

Allen Chen
Chenise Harper
Phillip Mason
Kareena Shah

6.4200

April 10, 2026

1 Introduction

Author(s): Kareena Shah

The objective of Lab 06 was to achieve two core elements of autonomous operation: **planning** and **control**. Autonomous operations, particularly of vehicles, are a primary topic of research unlocking increased efficiency and accessibility in transport.

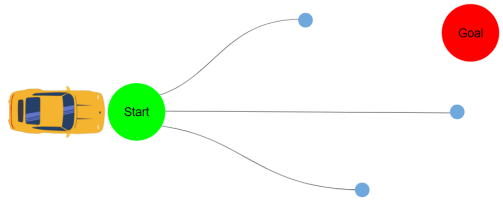

Figure 1: Autonomous operations of automotive vehicles unlocks new means of ground transport with increased accessibility and efficiency.

Firstly, we were tasked with implementing either a **grid-based** or **sampling-based** motion planning method to plan trajectories in a known occupancy grid map from the racecar's current position to a goal pose.

In grid-based planning, a discrete grid or graph is searched to find a feasible and minimum-cost path from the start to the end goal. However, inefficiencies occur when the complexity of the grid representation grows exponentially with set of all possible positions or poses of a system.

Alternatively, sampling-based planning involves randomly sampling points and creating paths between them until a solution is found or time expires. Sampling-based planners offer fast and efficient solving, but also might yield inefficient paths.

Our team was also able to implement both the grid-based and sample-based planning methods. This project also implements the particle filtering algorithm developed in Lab 05 for pose estimation along with a pure pursuit controller, a geometric path-tracking algorithm, to program the racecar to follow a predefined trajectory in a known occupancy grid map.

The combination of these factors enabled successful real-time path planning within simulation and on the physical racecar.

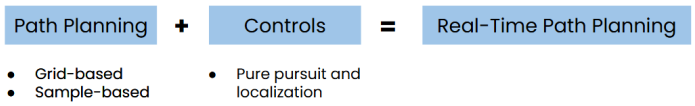

Figure 2: Real-time path planning is enabled through path planning implementation and integration of localization and pure pursuit control.

2 Technical Approach

Author(s): Allen Chen, Chenise Harper, Kareena Shah

To enable the autonomous vehicle to determine its exact position on a known map without relying on external tracking infrastructure (such as GPS), our team implemented a Monte Carlo Localization (MCL) algorithm. This probabilistic approach relies on matching real-time sensor data against a digital map to continuously estimate the vehicle's state.

Then, to allow the robot to plan a path between two points and follow the path without hitting obstacles, we implemented both sampling and grid-based path planning algorithms and a pure pursuit algorithm.

2.1 Monte Carlo Localization

Our localization system uses Monte Carlo Localization (MCL), a recursive Bayesian filtering method that estimates the robot pose using a distribution of weighted particles instead of a single deterministic state. The belief update is given by

$$bel(x_t) = \eta\, p(z_t \mid x_t) \int p(x_t \mid u_t, x_{t-1})\, bel(x_{t-1})\, dx_{t-1}, \tag{1}$$

where $bel(x_t)$ is the belief over the current pose, u_t is the odometry input, z_t is the LiDAR observation, and η is a normalization constant. Since evaluating this continuous integral exactly is too expensive for real-time operation, MCL approximates the belief distribution using a finite set of particles. Each update cycle consists of motion propagation, sensor weighting, and resampling.

The motion model approximates $p(x_t \mid u_t, x_{t-1})$ by moving each particle according to the odometry measurement while adding noise to represent physical uncertainty. Given local odometry motion $(dx, dy, d\theta)$, each particle is updated by rotating this local displacement into the global map frame:

$$x' = x + \hat{dx}\cos(\theta) - \hat{dy}\sin(\theta), \tag{2}$$

$$y' = y + \hat{dx}\sin(\theta) + \hat{dy}\cos(\theta), \tag{3}$$

$$\theta' = \theta + \widehat{d\theta}. \tag{4}$$

The noisy motion terms are sampled as

$$\hat{dx} = dx + \mathcal{N}(0, \sigma^2_{trans}), \quad \hat{dy} = dy + \mathcal{N}(0, \sigma^2_{slip}), \quad \widehat{d\theta} = d\theta + \mathcal{N}(0, \sigma^2_{rot}). \tag{5}$$

The three motion parameters α_{trans}, α_{slip}, and α_{rot} control the magnitude of this uncertainty. They are necessary because real vehicle motion is not perfectly captured by odometry: forward motion can be misestimated, the car can slip laterally, and heading can drift during turns. Larger movements introduce more uncertainty, so the noise scales with the magnitude of the measured motion:

$$\sigma_{trans} \propto \alpha_{trans}(|dx| + |dy|), \quad \sigma_{slip} \propto \alpha_{slip}(|dx| + |dy|), \quad \sigma_{rot} \propto \alpha_{rot}|d\theta|. \tag{6}$$

The sensor model approximates $p(z_t \mid x_t)$ by comparing the real LiDAR scan against expected raycasted measurements from the map. For each particle i and beam k, the expected range $d_k^{[i]}$ is compared to the observed range z_k. Instead of assuming a perfect match, the likelihood is modeled as a mixture of four terms:

$$P(z \mid d) = \alpha_{hit} p_{hit} + \alpha_{short} p_{short} + \alpha_{max} p_{max} + \alpha_{rand} p_{rand}. \tag{7}$$

Each parameter has a specific role. α_{hit} weights correct wall measurements, α_{short} accounts for unexpected short readings caused by unmapped obstacles, α_{max} handles maximum-range returns, and α_{rand} gives robustness to random sensor noise. The parameter σ_{hit} controls how tolerant the model is to small differences between the measured and expected ranges. A smaller σ_{hit} makes the filter stricter, while a larger value makes it more forgiving of map error and sensor noise. Our tuned

model used

$$P(z \mid d) = 0.82p_{hit} + 0.03p_{short} + 0.05p_{max} + 0.10p_{rand}. \tag{8}$$

To reduce computation, each scan was downsampled to $K = 99$ beams. The particle weight is computed from the beam likelihoods as

$$w^{[i]} = \prod_{k=1}^{99} \left(P(z_k \mid d_k^{[i]}) \right)^{1/3}. \tag{9}$$

The exponent of 1/3 acts as a squash factor. This is necessary because adjacent LiDAR beams are not truly independent, so directly multiplying all beam probabilities can make the filter overconfident and assign near-zero weights to particles that are only slightly misaligned. The squash factor flattens the distribution and improves robustness.

The particle filter is initialized around a user-provided pose estimate using a Gaussian distribution with standard deviation 0.7 for x and y and 0.5 for θ. After each motion and sensor update, particles are resampled according to their weights so that high-probability poses are preserved while unlikely poses are removed. A small Gaussian blur with standard deviation 0.1 for x and y and 0.05 for θ is added after resampling to maintain diversity and prevent particle collapse. Repeating this cycle allows the particle cloud to converge toward the robot's true pose while still maintaining enough uncertainty to recover from odometry drift and sensor noise.

2.2 Path Planning - Trajectory Planner

Path planning is the process of computing a safe, feasible route from the vehicle's current position to a desired goal while avoiding obstacles in the map. The planner takes the car's current pose, a goal pose from RViz, a simulation visualizer, and an occupancy grid map with inflated obstacles and walls, then outputs a collision-free trajectory in the map frame. Our path planner is able to generate a path using either a grid-based A* search or a sampling-based RRT* method, depending on the selected planning mode as both methods offered unique advantages and disadvantages.

2.2.1 Dilating Map Obstacles

The map arrives as an `OccupancyGrid` which gets reshaped into a 2D array of Cartesian coordinates that directly correspond to the map's cells. Morphological dilation is then applied to the occupied cells in order to inflate obstacles.

$$r_{\text{pixels}} = \left\lfloor \frac{r_{\text{inflate}}}{\text{resolution}} \right\rfloor$$

such that the obstacles take up extra r_{pixels} pixels around them given that there was a desired r_{inflate}, the desired radial padding in meters, and the resolution of the map. The effects of dilation are shown in Figure 3.

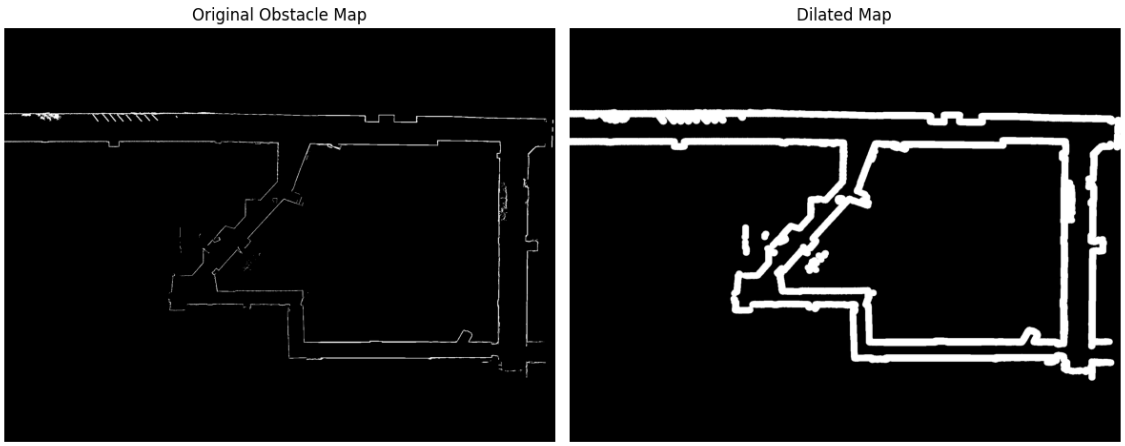

Figure 3: The map is converted to grayscale and the white cells are dilated to create the new map that will be used which keeps the planned path further away from obstacles. This figure used $r_{\text{inflate}} = 0.5$ with resolution of 0.0483 meters/pixel.

After dilation, any cells above the occupancy threshold, the ratio of a cell that contains an obstacle (0.5), as well as unknown cells, are treated as blocked, and all remaining cells are marked as free for planning.

2.2.2 Grid-Based Path Planning: A*

For grid-based path planning, we implemented A* because it provides a strong balance between path quality and computational efficiency on a 2D occupancy grid. We considered BFS, Dijkstra's algorithm, and A*. BFS is simple and complete, but it treats every motion as having equal cost, which is not ideal for an 8-connected grid where diagonal motion should cost more than horizontal or vertical motion. Dijkstra's algorithm improves on BFS by supporting weighted edge costs, so it can correctly assign straight moves a cost of 1 and diagonal moves a cost of $\sqrt{2}$. However, Dijkstra's algorithm expands outward uniformly from the start, which can be inefficient for larger maps. A* was chosen because it keeps Dijkstra's cost-aware structure while adding a goal-directed heuristic, allowing the planner to search fewer cells while still finding a low-cost collision-free path.

The robot's start position is obtained from the transform between the map frame and `base_link`, while the goal position is provided through RViz on `/goal_pose`. Both points are converted from world coordinates into grid coordinates using

$$
\begin{aligned}
dx &= x - x_{\text{origin}}, \\
dy &= y - y_{\text{origin}}, \\
l_x &= dx\cos(-\theta_{\text{map}}) - dy\sin(-\theta_{\text{map}}), \\
l_y &= dx\sin(-\theta_{\text{map}}) + dy\cos(-\theta_{\text{map}}), \\
u &= \left\lfloor \frac{l_x}{r} \right\rfloor, \\
v &= \left\lfloor \frac{l_y}{r} \right\rfloor.
\end{aligned}
$$

Here, (x, y) is the point in world coordinates, $(x_{\text{origin}}, y_{\text{origin}})$ is the map origin, θ_{map} is the yaw of the map frame, r is the map resolution, (l_x, l_y) is the point expressed in the local map frame, and (u, v) is the grid cell used by the planner. Before search begins, the planner checks that the start and goal cells are inside the map and are not blocked in the inflated grid.

A* then searches over the neighboring cells around the current cell. The four horizontal and vertical neighbors have cost 1, while the four diagonal neighbors have cost $\sqrt{2}$. Each candidate cell is ranked using cost function

$$
f(n) = g(n) + h(n),
$$

where $g(n)$ is the accumulated cost from the start to cell n, and $h(n)$ is the Euclidean-distance estimate from cell n to the goal. A priority queue is used so that the cell with the lowest $f(n)$ is expanded first. The implementation stores the lowest known cost to each cell in a `g_score` table and stores parent relationships in a `came_from` table. These two structures allow the planner to update a cell if a cheaper route is found and reconstruct the final path after the goal is reached.

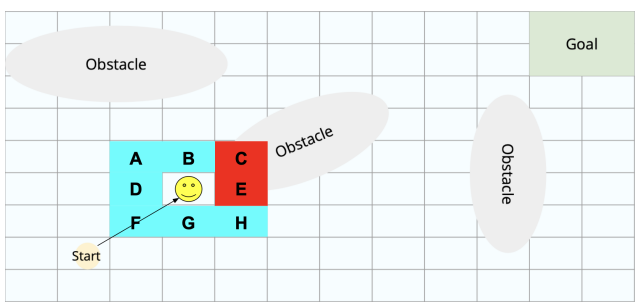

Figure 4: Example A* neighbor-expansion step on an occupancy grid. The robot, represented by a smiley face, starts at the beige circle and searches toward the green goal. The lettered cells represent candidate neighbors evaluated by the planner. Free blue cells are assigned motion costs, while occupied red cells are rejected. The planner expands the valid neighbor with the lowest total cost $f(n)$.

Figure 14 illustrates how the planner evaluates neighboring cells. For each valid neighbor, A*

combines the cost already traveled with the estimated remaining cost to the goal. For example, cell B reached through two diagonal moves and one straight up move with accumulated cost $g(n) = 1 + 2\sqrt{2}$. Since cell B is three grid cells up and seven grid cells right from the goal, the Euclidean heuristic is $h(n) = \sqrt{3^2 + 7^2} = \sqrt{58}$. The total cost is $f(n) = g(n) + h(n) = 11.44$. This is also the cell with the lowest cost, so it will be explored first.

The implementation also includes a diagonal corner-cutting check. A diagonal move is only allowed if the adjacent horizontal and vertical cells are both free. This prevents the planner from producing paths that squeeze between two obstacle corners, which would be mathematically valid on the grid but physically unsafe for the car.

Once the goal cell is reached, the planner reconstructs the path by following the `came_from` table backward from the goal to the start.

2.2.3 Sampling-Based Planning: RRT*

Sampling-based motion planning algorithms are widely used in robotics due to their efficiency in high-dimensional spaces.

RRT*, or Rapidly-exploring Random Trees*, was used for this sampling-based implementation. At a high level, RRT* builds a tree of possible paths through random sampling and improves upon it through cost optimization.

Points are sampled within the configuration space, C, which is the set of all possible states. Particularly, each sampled point is randomly sampled within the free space configuration (C_{free}) which consists of valid states where motion is allowed. They each have a parent and cost associated with it.

$$x_{rand} \in C_{free}$$

The algorithm starts with a single node as the starting point which has no parent or cost associated with it.

Figure 5: The sample planner starts with only the starting node as shown by the yellow circle.

Each point is sampled using one of three random sampling methods: uniform sampling, bridge sampling, and goal biasing.

Uniform sampling involves independent random sampling of points within the free configuration space as seen in Figure 6.

Figure 6: In uniform sampling random samples are drawn independently from the free configuration space.

Bridge sampling randomly generates three sample points, two of which lie on or near obstacles as represented in Figure 7. If the midpoint lies in free space, it is added as a valid sample. Bridge sampling is designed to improve exploration in narrow passages.

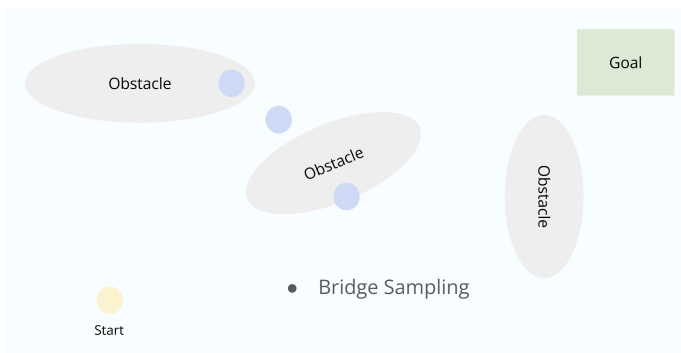

Figure 7: In bridge sampling groups of three sample points are randomly such that at least two lie on or near obstacles.If the midpoint lies in free space, it is added as a valid sample.

Goal Biasing increases the probability of sampling the goal state directly:

$$x_{rand} = \begin{cases} x_{goal}, & \text{with probability } p \\ \text{Uniform}(C_{free}), & \text{otherwise} \end{cases}$$

where p is the goal bias probability which has been set to 0.1. This technique accelerates convergence toward the goal by encouraging the tree to grow in its direction which can be seen in Figure 8.

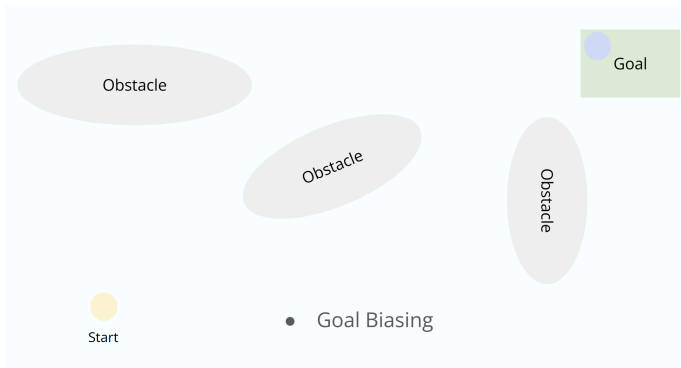

Figure 8: After a certain number of sampled points are generated, goal biasing is employed to ensure that the generated tree tends towards the intended goal state by sampling directly on the final desired location.

After sampling a point using one of these three methods, the car finds the nearest node in the tree.

$$x_{nearest} = \arg\min_{x \in V} \|x - x_{rand}\|$$

It then steers towards the sample point by moving a small amount in the direction of the point as shown in Figure 9. δ is the maximum step size (set to 0.5).

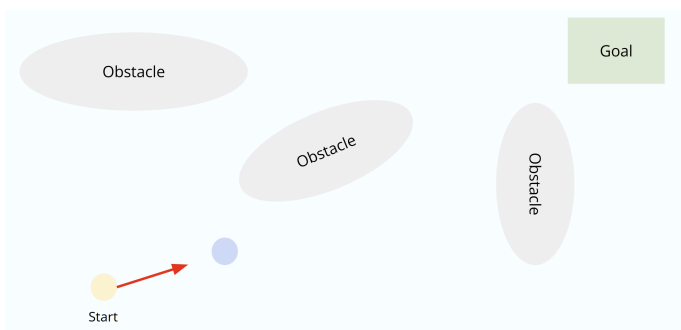

Figure 9: The car steers towards the sample point by finding the nearest node and moving a small δ amount in its direction.

The new node x_{new} is computed as:

$$x_{\text{new}} = \begin{cases} x_{\text{nearest}} + \delta \dfrac{x_{\text{rand}} - x_{\text{nearest}}}{\|x_{\text{rand}} - x_{\text{nearest}}\|}, & \text{if } \|x_{\text{rand}} - x_{\text{nearest}}\| > \delta \\ x_{\text{rand}}, & \text{otherwise} \end{cases} \tag{10}$$

The cost is calculated as a function of path length. For a node x, the cost function is given by:

$$J(x) = \sum_{i=1}^{k} \|x_i - x_{i-1}\| \tag{11}$$

where $\{x_0, x_1, \ldots, x_k = x\}$ is the sequence of nodes forming the path from the start node x_0 to node x, and $\|\cdot\|$ denotes the Euclidean norm.

When a new node x_{new} is generated from its nearest neighbor x_{nearest}, the initial cost is computed as:

$$J(x_{\text{new}}) = J(x_{\text{nearest}}) + \|x_{\text{new}} - x_{\text{nearest}}\| \tag{12}$$

Upon calculating the cost associated with the new node, RRT* selects a new parent by searching for a neighboring node that minimizes the cost:

$$J(x_{\text{new}}) = \min_{x_i \in \mathcal{N}} [J(x_i) + \|x_{\text{new}} - x_i\|] \tag{13}$$

RRT* also checks for obstacle collisions before adding a new node as seen in Figure 10. Points along a straight-line segment between the nearest node and the sampled point are checked to ensure that no obstacles lie in between or on the sampled point. If an obstacle is detected, the sampled point is discarded and a new point is resampled.

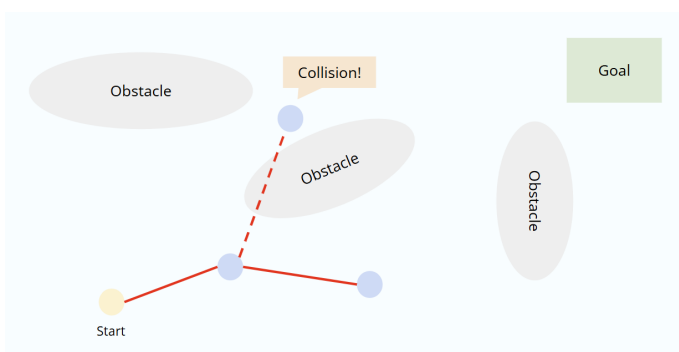

Figure 10: RRT* checks for potential collisions with obstacles in the space by checking points along a straight-line segment between the nearest node and newly sampled point.

After adding x_{new}, nearby nodes are checked to determine if their cost can be reduced by rewiring through x_{new}:

$$J(x_i) > J(x_{\text{new}}) + \|x_i - x_{\text{new}}\| \tag{14}$$

If this condition holds, the parent of x_i is updated to x_{new}. This ensures that the tree constantly improves and the best path is optimized by minimizing the associated cost.

The tree generated by RRT* tends to grow toward unexplored portions in the sample space as seen in Figure 11.

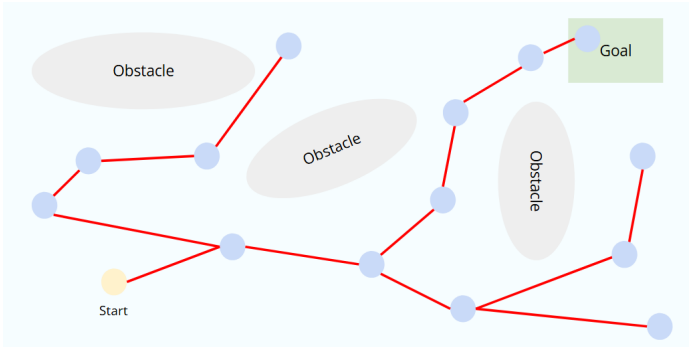

Figure 11: With RRT*, a growing tree is generated where points are sampled throughout the environment. It enables exploration of previously unexplored portions of the sample space, further ensuring that an optimal solution is reached.

The distance between a new node and the goal point is constantly calculated as well. If the distance is within the designated threshold distance of the goal, the total cost to reach the goal is computed as:

$$J(x_{\text{goal}}) = J(x_{\text{new}}) + \|x_{\text{goal}} - x_{\text{new}}\| \tag{15}$$

As indicated in Figure 12, planning process is terminated and the lowest-cost path is selected when the goal is reached.

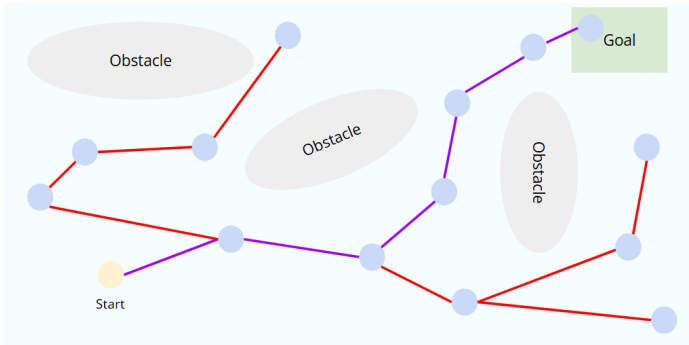

Figure 12: Final search tree of RRT* algorithm, where the purple path represents the final trajectory path and the red lines are potential paths that were not finished before termination.

2.2.4 Grid-Based vs. Sampling-Based Planning

The choice between grid-based and sampling-based sampling depend on the following factors:

- **Map dimensionality:** Grid-based methods scale poorly with higher dimensions, while sampling-based methods handle high-dimensional spaces efficiently.
- **Optimality requirements:** Whether an optimal or near-optimal path is required.
- **Computation time constraints:** Availability of strict time limits for planning.

- **Environment structure:** Presence of narrow passages, clutter, or large open areas.
- **Motion constraints:** Whether vehicle dynamics or nonholonomic constraints must be incorporated.
- **Path quality vs feasibility:** Tradeoff between smooth, realistic paths and guaranteed shortest paths.

Grid-based planning (A*):
Grid-based methods discretize the environment into a fixed structure and perform graph search. As discussed earlier, A* is resolution-optimal and guarantees the lowest-cost path under the defined grid cost model when using an admissible heuristic. This makes it well-suited for environments where optimality is important.

However, grid-based planning has several limitations. The computational complexity increases significantly with map size and resolution, making it less scalable. The resulting paths are constrained to the grid, often leading to jagged or unnatural trajectories. Furthermore, it does not inherently account for vehicle dynamics unless additional constraints are introduced.

Sampling-based planning (RRT*):
Sampling-based methods such as RRT* operate directly in continuous space by randomly sampling valid configurations. These methods are probabilistically complete, meaning they will find a solution with high probability given enough time. RRT* is also asymptotically optimal, so the solution improves as more samples are added.

The main advantage of RRT* is its ability to efficiently explore large or complex spaces, including those with narrow passages or higher dimensions. It can also more naturally incorporate motion constraints and produce smoother, more feasible paths.

However, RRT* does not guarantee an optimal solution within finite time and can produce inconsistent or inefficient paths depending on sampling. It may also require careful tuning of parameters such as step size, goal bias, and sampling strategy. Most importantly, because the solution quality depends on the number of samples, it may struggle to meet strict time constraints.

Conclusion:
For this project, grid-based A* is the more suitable choice. The environment is given as a 2D occupancy grid with relatively low dimensionality, and the lab explicitly requires reliable path generation within 30 seconds. A* provides deterministic performance, guarantees an optimal path under the grid model, and is sufficiently fast for this scale of problem, which will show be shown later when we compare the real results produced by both planning algorithms during experimentation.

2.3 Pure Pursuit - Trajectory Follower

The trajectory tracking module is implemented as a ROS2 node (`/trajectory_follower`) utilizing the Pure Pursuit algorithm. This geometric path-tracking controller calculates the required steering angle to drive a vehicle toward a continuously updating lookahead point on a reference path.

2.3.1 System Overview

The `/trajectory_follower` node operates using a fixed-speed kinematic model and relies on three primary parameters:

- **Lookahead Distance (L_d):** 0.9 meters
  This is a circle of radius L_d and is "what the car can see".
- **Speed (v):** 1.0 m/s
- **Wheelbase Length (L_w):** 0.33 meters
  The approximate distance between the front and rear axle of the racecar.

The system architecture is event-driven, responding to incoming `Odometry` messages to trigger control loop updates, provided a valid `PoseArray` trajectory has been received. Control outputs are published as `AckermannDriveStamped` messages.

2.3.2 Vehicle Localization and Frame Transformation

Upon receiving an odometry update, the vehicle's current global position (x, y) and quaternion orientation are extracted. The quaternion is converted to a standard Euler yaw angle (θ) representing the vehicle's heading.
Once a target lookahead point (x_t, y_t) is identified in the global frame, it must be transformed into the vehicle's local coordinate frame. This is achieved by translating the global origin to the vehicle's center and applying a 2D rotation matrix by the negative yaw angle ($-\theta$):

$$x_l = \cos(\theta)(x_t - x) + \sin(\theta)(y_t - y) \tag{16}$$

$$y_l = -\sin(\theta)(x_t - x) + \cos(\theta)(y_t - y) \tag{17}$$

Where (x_l, y_l) represent the target point's lateral and longitudinal coordinates relative to the vehicle's `base link` frame.

2.3.3 Steering Angle Calculation

The core of the Pure Pursuit algorithm calculates the curvature (κ) of an arc connecting the vehicle's rear axle to the target point. Under the Ackermann steering model, the steering angle (δ) required to follow this arc is geometrically derived from the target point's local y-coordinate (y_l) and the square of the actual lookahead distance ($L_d^2 = x_l^2 + y_l^2$).

The control law implemented is:

$$\delta = \arctan\left(\frac{2L_w y_l}{L_d^2}\right) \tag{18}$$

To respect the physical constraints of the steering hardware, the resulting angle δ is clipped at $\pm 20°$ (± 0.34 radians) before being published to the drive topic.

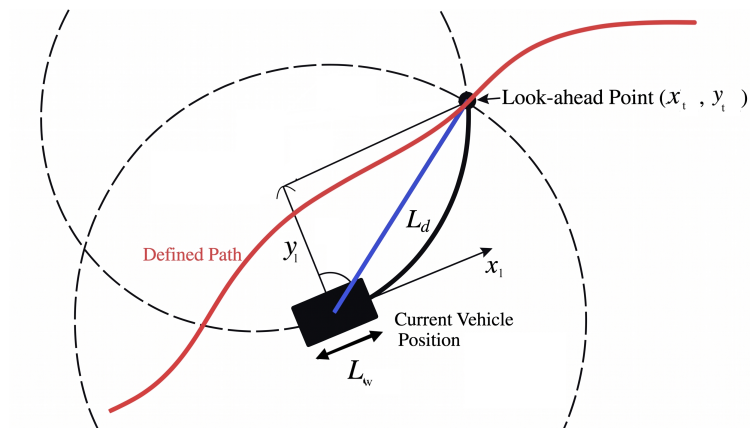

Figure 13: Geometric derivation of the Pure Pursuit steering angle, illustrating the relationship between the vehicle's wheelbase (L_w), the lookahead distance (L_d), and the target lateral offset (y_l).

2.3.4 Lookahead Point Intersection Algorithm

To find the target point (x_t, y_t), the algorithm computes the intersection between a circle of radius L_d centered at the vehicle (x, y) and the piecewise linear segments of the trajectory provided.
For a given segment defined by points $\mathbf{v}_1$ and $\mathbf{v}_2$ (shifted to the vehicle's local origin), the direction vector is $\mathbf{d} = \mathbf{v}_2 - \mathbf{v}_1$. The intersection is parameterized by t, found by solving the quadratic equation $at^2 + bt + c = 0$, where:

$$a = \mathbf{d} \cdot \mathbf{d} \tag{19}$$

$$b = 2(\mathbf{v}_1 \cdot \mathbf{d}) \tag{20}$$

$$c = (\mathbf{v}_1 \cdot \mathbf{v}_1) - L_d^2 \tag{21}$$

The algorithm evaluates the discriminant ($\Delta = b^2 - 4ac$). If $\Delta \geq 0$, the segment intersects the circle. The valid parameters t_1 and t_2 are constrained such that $0 \leq t \leq 1$ to ensure the intersection lies strictly on the segment.
If multiple valid intersections exist across the trajectory, the algorithm iteratively updates the target point. To prevent the vehicle from targeting points behind it, the vector to the intersection point $(\Delta x, \Delta y)$ is verified against the vehicle's heading using a dot product:

$$\cos(\theta)\Delta x + \sin(\theta)\Delta y > 0 \tag{22}$$

2.3.5 Goal Tolerance and Edge Cases

To ensure stable termination, the node continuously monitors the Euclidean distance between the vehicle's current position and the final point in the trajectory array.

- **Stopping Criterion:** A physical tolerance of 0.15 meters is enforced. Because continuous physical systems rarely achieve absolute zero error, triggering a stop at 15 cm prevents oscillatory behavior around the terminal waypoint.

- **Missing Lookahead Points:** If no path segments intersect the lookahead circle, but the final goal is physically within the lookahead radius ($d \leq L_d$), the vehicle overrides the search algorithm and targets the final goal directly. If the path is entirely lost, a fallback mechanism stops the movement of the racecar.

3 Experimental Evaluation

Author(s): Allen Chen, Phillip Mason

To ensure our implementation was accurate, the localization algorithm was re-evaluated using ground truth and our path planning and following were evaluated through testing speed, path distance and recording error as the path is followed. Our localization had an error of 14.41% and our paths generated in under 30 seconds and were followed within 0.5 meters.

3.1 Evaluating our Localization on Real Ground Truth

In Lab 05, our sensor and motion model parameters for localization were the following.

Table 1: Summary of the main localization parameters.

Module	**Parameter**	**Value**	**Description**
Motion Model	α_{trans}	0.10	Controls forward translational noise. Larger values spread particles more in the direction of travel.
	α_{rot}	0.05	Controls rotational noise. Larger values create more uncertainty in particle heading.
	α_{slip}	0.02	Controls lateral slip noise. Larger values spread particles sideways.
Sensor Model	α_{hit}	0.85	Weight for expected lidar measurements that match the map.
	α_{short}	0.02	Weight for unexpectedly short lidar readings.
	α_{max}	0.05	Weight for max-range lidar readings.
	α_{rand}	0.08	Weight for random lidar measurements and sensor noise.
Particle Filter	`num_particles`	100	Number of particles used to represent the belief distribution.
	`init_x_std`	0.7	Initial spread of particles in the x direction.
	`init_y_std`	0.7	Initial spread of particles in the y direction.
	`init_theta_std`	0.5	Initial spread of particles in heading.
	`resample_position_std`	0.2	Position noise added after resampling to maintain particle diversity.
	`resample_theta_std`	0.1	Heading noise added after resampling to maintain orientation diversity.

In order to evaluate our localization to real ground truth, we decided to measure the real lengths of sections of the Stata basement loop.

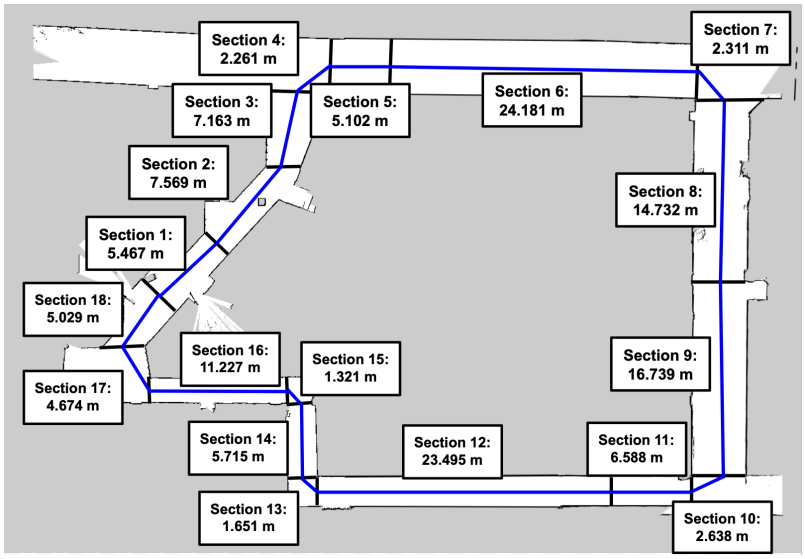

Figure 14: The map of Stata basement is split up by black lines into 18 sections based on noticeable landmarks. The blue lines mark the distances that were measured.

For each of the specified sections, we drove the racecar from one black line to another and recorded the estimated pose from the particle filter. We then calculated the estimated distance driven using the first and last pose from particle filter for that section using Euclidean geometry and compared it to the actual tape measurements shown above in Fig. 14.

Table 2: Localization section distance evaluation. The absolute distance error is computed as $e = |d_{\text{est}} - d_{\text{real}}|$, and the percentage error is computed as $e_{\%} = \frac{|d_{\text{est}} - d_{\text{real}}|}{d_{\text{real}}} \times 100$, where d_{est} is estimated distance calculated using the first and last pose of /pf/pose/odom and d_{real} is the real distance measured.

Error equations: $e = \lvert d_{\text{est}} - d_{\text{real}} \rvert, \quad e_{\%} = \frac{\lvert d_{\text{est}} - d_{\text{real}} \rvert}{d_{\text{real}}} \times 100$				
Section	**Estimated Distance (m)**	**Real Distance (m)**	**Error (m)**	**Error (%)**
1	3.7600	5.4670	1.7070	31.22
2	5.2398	7.5690	2.3292	30.77
3	5.6956	7.1630	1.4674	20.49
4	2.4429	2.2610	0.1819	8.04
5	4.6782	5.1020	0.4238	8.31
6	21.9557	24.1810	2.2253	9.20
7	2.1028	2.3110	0.2082	9.01
8	12.7760	14.7320	1.9560	13.28
9	14.6540	16.7390	2.0850	12.46
10	2.7968	2.6380	0.1588	6.02
11	5.1293	6.5880	1.4587	22.14
12	20.0345	23.4950	3.4605	14.73
13	1.5554	1.6510	0.0956	5.79
14	4.7378	5.7150	0.9772	17.10
15	1.2640	1.3210	0.0570	4.32
16	9.6524	11.2270	1.5746	14.03
17	4.1884	4.6740	0.4856	10.39
18	3.9170	5.0290	1.1120	22.11
		Mean	**1.2202**	**14.41**

The localization results show that the particle filter generally follows the correct route, but it often underestimates the traveled distance. The mean distance error was 1.2202 m, with a mean percentage error of 14.41%.

Sections 1, 2, and 3 had the largest early errors, suggesting possible initialization or convergence issues. Sections 6 and 12 should also be explored because they had large absolute errors over long distances, indicating possible accumulated heading or odometry drift. Lower-error sections, such as 4, 10, 13, and 15, show that the filter works well when the map geometry is distinctive. Overall, the particle filter is functional, but performance depends strongly on sensor correction, motion-model tuning, and map uniqueness.

3.2 Path Planning

The grid-based and sample-based path planning algorithms were both able to plan paths around the Stata basement. With our obstacle inflation radius, they also stay far enough away from the walls for the car to follow. A notable difference shown in Figure 15 is how optimal and straight

the grid path is, as oppose to how the sample path changes direction often and sometimes takes inefficient angles straying away from the goal.

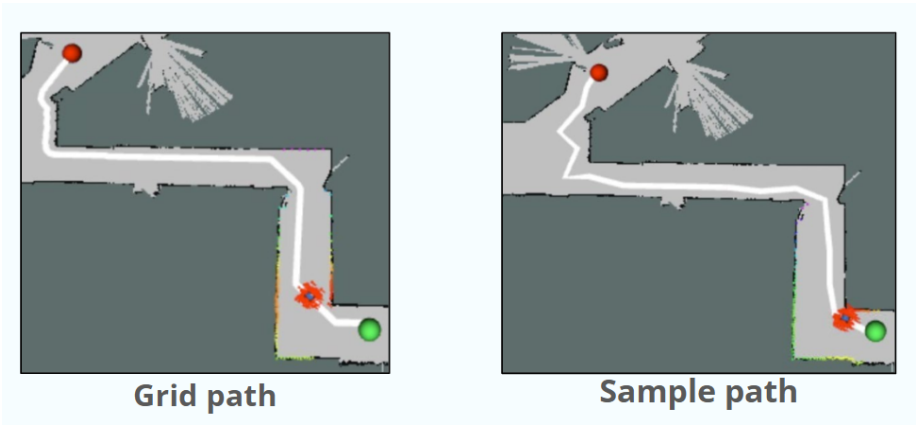

Figure 15: The grid path is much more smooth and straight, while the sample path frequently has small angle adjustments that sometimes lead further from the goal than continuing straight would.

As mentioned earlier, the grid-based path planning is more suited for our purposes with a much shorter generation time. By generating roughly the same paths as in Figure 15 3 times for grid-based and sample-based planning, it is clear through Table 3 how much faster A* is. RRT* would be more efficient on a larger and more complex scale, but sticking to the Stata basement, this is irrelevant. Still, both stay well below the design requirement of 30 seconds.

Table 3: Statistics of Generated Paths

Test	Grid 1	Grid 2	Grid 3	Sample 1	Sample 2	Sample 3
Path Generation Time (s)	0.67	0.41	0.53	9.41	12	18
Path Distance (m)	23.1	23	23.2	23.63	20.3	22.3

3.3 Path Following

The racecar used pure pursuit to accurately follow its generated path within 0.5 meters. Our initial lookahead distance was too high, causing the car to turn early and stop right before hitting corners. This distance was then tuned to be small enough for turns while being big enough to maintain a smooth path. Figure 16 shows that the car remains within 0.5 meters of the path, with an average error of 0.118 meters. Two similar trials depicted in the Appendix had average errors of 0.119 meters and 0.138 meters, and all 3 trials stay below 0.5 meters, showing the car's path following accuracy.

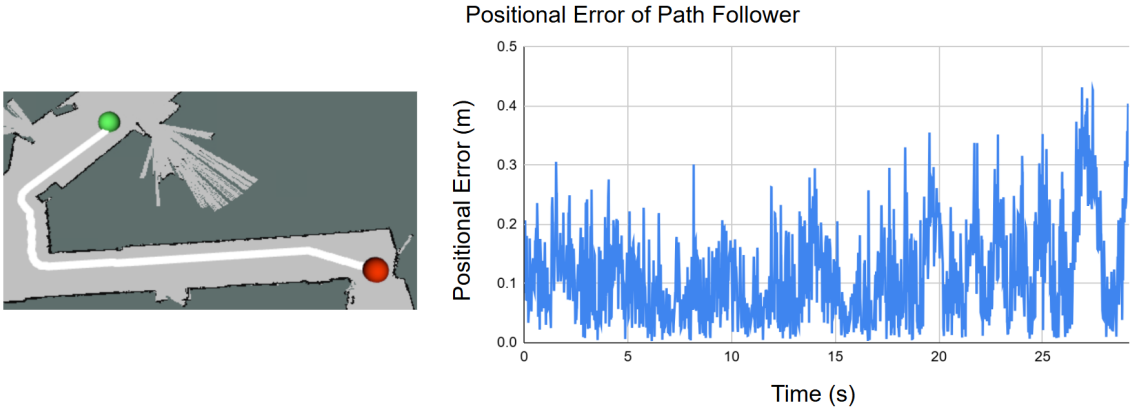

Figure 16: **The car remains within 0.5 meters of the path, with an average error of 0.118 meters, showcasing our pure pursuit algorithm's accuracy.** On the left is the path followed, and on the right is the distance between the car and the path over time.

4 Conclusion

Author(s): Phillip Mason

With the algorithms created in this lab, the robot is now able to plot a trajectory from its current position to any point published on a known map while avoiding obstacles. While RRT* would be more flexible for higher dimension environments, A* quickly generates smooth paths in a structured setting like our car navigating Stata basement. After implementing both A* and RRT* path planning, it was clear that A* had more optimal paths and faster generation for our purposes.

The car can then reliably follow this trajectory within 0.5 meters using the MCL localization developed in Lab 05 alongside a pure pursuit controller. Future improvements include tuning the pure pursuit controller for higher speeds and dynamic obstacles. The final challenge will also require building on this for a full lap of the basement with intermittent parking.

4.1 Lessons Learned

4.1.1 Allen Chen

This lab taught me that:

- Sometimes skipping to other parts of the project can help increase morale and get out of a stalemate and then come back later with new ideas.
- Planning when to work on the robot weekly and to have some emergency work times every week is necessary as I cannot predict some things happening in my personal life.

4.1.2 Chenise Harper

By completing this lab, aside from technical knowledge, I learned that:

- I like doing this kind of work. I am grateful for all of the skills that I am learning in this class and I look forward to continuing work in robotics and efficiently communicating my lessons and discoveries through professional narratives like we have been practicing.
- Hardware constraints are very difficult to ignore. In this lab taking things like the distance between the wheels in calculations and efficiency for performance on the car has been very humbling and important. I find it fascinating the ways that the hardware components interact with our software and visualizing tools and looking more into these constraints for the final project will be very fun.

4.1.3 Phillip Mason

This lab taught me how to communicate my findings more professionally, which will be useful far past this class for my eventual career.

- It is important to discuss details for my graphs before they appear in the paper, but the captions should also be detailed enough that someone who only reads the images and captions still gets the full picture.
- I should lead with my findings and then elaborate on them, repeating them often so that they are unmissable and researchers reading my paper don't have to comb through thoroughly searching for my results.
- I have also learned how to compare two similar algorithms and empirically justify which one better suits a specific scenario.

4.1.4 Kareena Shah

This final lab taught me a multitude of things about communication, integration, and analysis.

- I learned about the importance of communicating technical implementation decisions prior to development to ensure seamless integration of independent modules in later stages. In particular, this was especially important when putting the planning and controls aspects of the lab together. In doing so, we were able to set a standard among our team, which minimized the potential for integration issues and helped us gain more time for testing. This also ensured that the team is on the same page and that everyone had the same expectations.
- I also learned about the importance of conducting trade studies. Since we were able to dig deeper into two unique path planning algorithms, we also had to gain a better understanding of how they worked and conduct cost-benefit analyses to see which implementation would be better suited for our purposes in this class. In this way, we were able to make informed decisions about our final path planner implementation, which helped us improve our overall performance, thus setting us up well for the final challenge.

5 Appendix

5.1 Sensor Model Probabilities

1. **True Hit (p_{hit}):** Accounts for accurate measurements, applying a Gaussian curve to absorb minor map inaccuracies. It is defined by the sensor's variance ($\sigma_{hit} = 3.0$ pixels):

$$p_{hit}(z \mid d) = \exp\left(-\frac{(z-d)^2}{2\sigma_{hit}^2}\right) \tag{23}$$

2. **Short Hit (p_{short}):** Models unexpected dynamic obstacles (e.g., pedestrians) that obstruct the laser beam before it reaches the mapped wall:

$$p_{short}(z \mid d) = \begin{cases} \frac{2}{d}\left(1 - \frac{z}{d}\right) & \text{if } 0 \leq z \leq d \\ 0 & \text{otherwise} \end{cases} \tag{24}$$

3. **Max Range (p_{max}):** Accounts for sensor saturation or beams absorbed by non-reflective surfaces, returning the maximum range limit (z_{max}):

$$_{max}(z \mid d) = \begin{cases} 1 & \text{if } z = z_{max} \\ 0 & \text{otherwise} \end{cases} \tag{25}$$

4. **Random Noise (p_{rand}):** Captures completely unexplained phantom readings or LiDAR crosstalk via a uniform distribution:

$$p_{rand}(z \mid d) = \begin{cases} \frac{1}{z_{max}} & \text{if } 0 \leq z < z_{max} \\ 0 & \text{otherwise} \end{cases} \tag{26}$$

5.2 Graphs for Evaluation of Pure Pursuit Path Follower

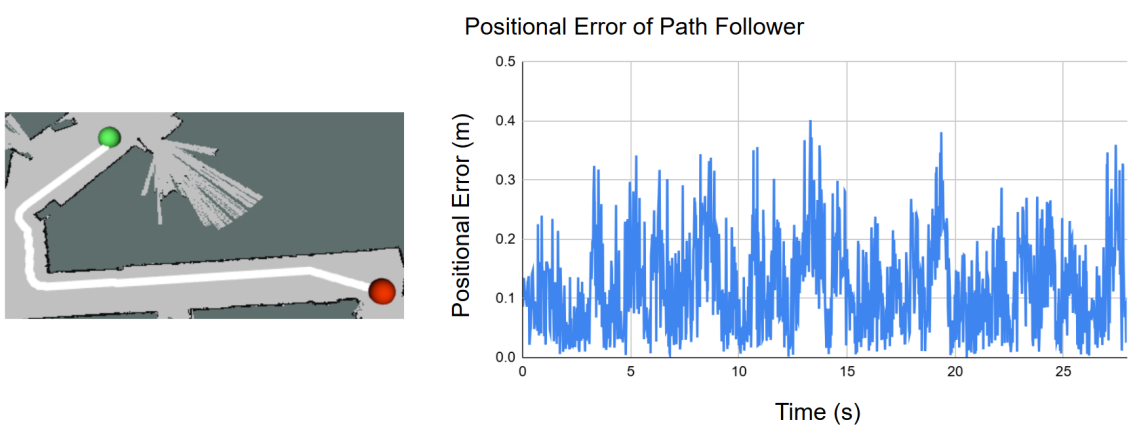

Figure 17: Path and Positional Error over time of Second Trial. Average Error = 0.119 meters

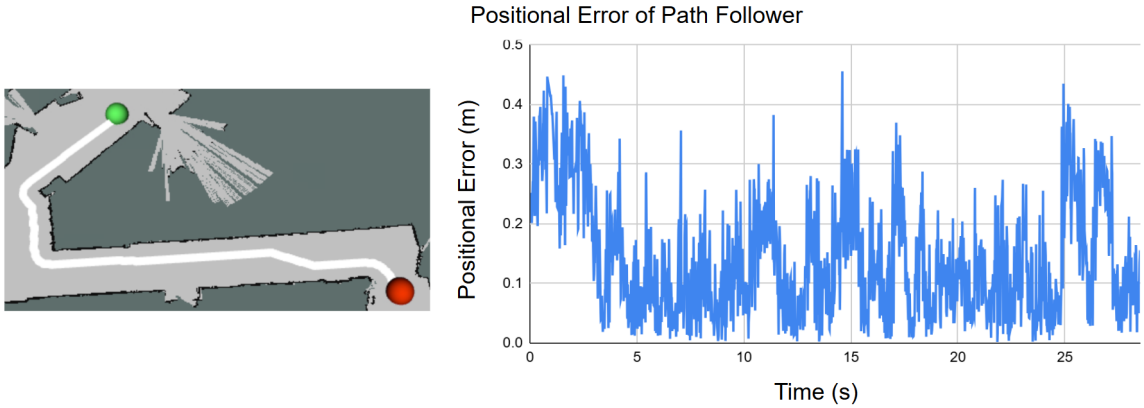

Figure 18: Path and Positional Error over time of Second Trial. Average Error = 0.138 meters